# A Smaller Implementation and a Deterministic Closure Presentation for iPRA

Research note prepared for Vladimir Reshetnikov

10 November 2026

**Abstract**

The requested strictly simpler mathematical axiomatization has not been established. The defensible results are narrower. First, a local change to two complete Python proof checkers for the existing native scalar calculus reduces the inspected byte minimum from 529 to 528 and the inspected AST minimum from 289 to 288, in different programs. The accepted certificates are unchanged in the ideal resource-unbounded reading. Token counts and gzip sizes increase in the matched comparisons.

Second, a different native presentation has a deterministic introduction rule: literal-equation closures express progress, while compound closures retain the original conditional-universal meaning. Compound padding gives a proof translation from the existing core, and explicit formula composites establish a whole-domain bi-interpretation with iPRA. Its more elaborate elimination rule has not been shown to reduce total complexity. Finally, an exact zero-set theorem explains why fixing every scalar iterator's initial state to zero cannot preserve the existing term compiler. Complete rules, written proofs, executable checks, provenance, and the inherited arithmetic bridge are supplied.

## Contents

**1 Outcome, comparison target, and scope** **3**

1.1 The common arithmetic and logical baseline . . . . . 3

**2 The smaller complete implementations** **4**

2.1 Measurements and the unchanged public contract . . . . . 4

2.2 The exact transformation . . . . . 4

2.3 Executable evidence and provenance . . . . . 5

2.4 Complete byte-profile source . . . . . 6

**3 How to use the package and what remains open** **6**

**4 An alternative with deterministic, matrix-sensitive closure** **7**

4.1 Complete native presentation . . . . . 7

4.2 Induction and the logical padding operation . . . . . 8

4.3 An embedding of the existing native scalar core . . . . . 10

4.4 Ordinary interpretation and both composites . . . . . 10

**5 Why fixing the initial state to zero is not a free simplification 12**

**Attribution and dependency boundary 15**

**A Complementary introductions with one local assumption 15**

A.1 Complete native syntax and rule inventory . . . . . 15

A.2 Exact equivalence and the unused-witness construction . . . . . 16

A.3 Induction and numerical cases separate cleanly . . . . . 17

A.4 The entire ordinary logical interface . . . . . 18

A.5 Whole-domain realizations and both formula composites . . . . . 19

A.6 What the theorem does and does not claim . . . . . 20

**B Arithmetic strength and both ordinary interpretation composites 21**

B.1 The arithmetic dependency order . . . . . 21

**C Internal scalar predecessor bootstrap 22**

C.1 Elementary terms and two finite cycles . . . . . 23

C.2 A ramp with a global scaling identity . . . . . 24

C.3 A comparator from scalar iteration . . . . . 24

C.4 Saturated iteration gives predecessor . . . . . 25

C.5 Dependency boundary . . . . . 26

**D From literal-equation induction to full intuitionistic PRA 27**

D.1 The exact hypotheses . . . . . 27

D.2 Constructor separation and two elementary tests . . . . . 28

D.3 Four elementary uses of equation induction . . . . . 29

D.4 An equation invariant that reflects actual equality . . . . . 29

D.5 Compilation of all quantifier-free formulas . . . . . 31

D.6 Recovery of induction and the full theory . . . . . 32

**E Pairing, primitive recursion, and internal numerical composites 32**

E.1 Pairing from a bounded square-root search . . . . . 32

E.2 Compiling each primitive recursion . . . . . 33

E.3 Why both numerical round trips are internal . . . . . 34

# 1 Outcome, comparison target, and scope

The comparison target is the current `ConciseIPRA` project at commit `8662bc0a00f4`. The full commit is recorded in the package provenance. Its README describes the native scalar core and 22 filed research packages, with complete standalone-checker measurements and an ongoing Lean development [1]. All comparisons in this note use that fixed snapshot; they do not make a claim about later changes or a global minimum.

| Item | Established status |
|---|---|
| Existing-core implementation | Smaller by one byte or one AST node in separate complete programs; no change to the calculus. |
| Deterministic mixed closure | A different presentation, with written proofs of both interpretations and their composites. No smaller checker for it has been demonstrated. |
| Zero-start iterator deletion | A same-domain term-definability obstruction; not an impossibility theorem for arbitrary interpretations. |
| Requested axiomatic improvement | Still unresolved by this investigation. |

The distinction between these outcomes matters. Merely displaying one introduction schema can move its cost into another rule. Likewise, a one-byte implementation change does not eliminate an axiom or reduce arithmetical strength. This note reports the exact improvement it can justify and leaves the stronger requested conclusion open.

## 1.1 The common arithmetic and logical baseline

The current numerical terms are

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h),$$

where only the update body $h$ binds $u$. There are two computation schemas:

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The native formula vocabulary uses equality and the binders $Q, D$, with ordinary readings

$$Q_y B(y) = B(0) \to \forall y B(y), \qquad D_{x,y} B(x, y) = \exists x\, Q_y B(x, y).$$

The designated contradiction is $0 = S0$. The core has hypotheses, Leibniz transport through arbitrary native formula templates, conditional universal elimination, fresh-witness elimination, and the two introduction patterns

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}, \qquad \frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash D_{x,y} E(x, y)},$$

where the latter requires a literal equation and $z$ is fresh. The full inventory and all eigenvariable conditions are reproduced in Appendix A; that equivalent presentation treats $Q$ assumptions as reusable conditional clauses. This is inherited mathematics, not a new reduction.

The ordinary realization $T_{\mathrm{eq}}$ has intuitionistic first-order equality logic, the same iterator schemas, numerical coverage, $(0 = S0) \to \bot$, and contextual induction for literal equations. The inherited bridge constructs predecessor, equality reflection, quantifier-free induction, pairing, and a compiler for each fixed primitive-recursive definition. It proves both internal numerical composites, rather than only agreement in the standard model. The relevant proofs are reproduced in the appendices and retain their source attribution. In particular, this note does not replace full first-order iPRA with its equational or closed-numeral fragment.

## 2 The smaller complete implementations

### 2.1 Measurements and the unchanged public contract

The complete module is counted: JSON input, substitution/opening, contexts, rule checking, recursion, and the final verdict. The common infrastructure is CPython and its standard JSON library, exactly as in the comparison sources. A run accepts precisely when its exit status is zero and its standard output is the exact line `ACCEPT`. Uncaught checking or parsing exceptions reject. Neither a hidden checker import nor an external executable implements any proof rule.

Measurements use CPython 3.12.14. Parse the whole file with `ast.parse`, normalize with `ast.unparse`, and append one newline. Significant tokens exclude comments, indentation, line breaks, and stream markers; AST nodes are all nodes visited by `ast.walk`. Gzip uses compression level 9 and `mtime=0`; it is an informational coordinate. The two candidate source files are already in normalized form.

| Complete program | Tokens | AST nodes | Bytes | Gzip 9 |
|---|---|---|---|---|
| Reference AST profile | 263 | 289 | 613 | 349 |
| New AST profile | 264 | **288** | 608 | 350 |
| Reference byte profile | 266 | 310 | 529 | 333 |
| New byte profile | 267 | 309 | **528** | 336 |

The references are respectively `coverage_and_kernel/code/checker_ast.py` and `neutral_scope/code/checker.py` [3, 2]. The candidate modules are `code/checker_ast.py` and `code/checker_bytes.py`. The old 261-token coordinate remains unbeaten. No one delivered program has all three coordinate minima. The gzip figures increase in both matched comparisons. These are measurements of particular files, not lower bounds on possible checkers.

### 2.2 The exact transformation

An instruction checks a major premise, optionally changes the current goal by transport or closure specialization, computes an available hypothesis, and opens the next scope. The references execute the major check as a standalone statement:

```
c(m, g, k)
# Pure transport/specialization precheck and possible goal replacement.
h = m * 3 or ...
```

The new versions execute that pure precheck first and place the same major check at the start of the existing expression:

```
# The same pure precheck and possible goal replacement.
h = c(m, g, k) or m * 3 or ...
```

Every successful invocation of `c` returns `None`, so the inserted operand does not change the value of `h`. The major proof is still checked before that value is constructed. Crucially, the later scope-floor decrement is not moved. Witness opening retains its original order relative to fresh-variable allocation.

The idea of prevalidating an elimination template is already present in the repository's `prevalidated_elimination` package [4]. The contribution here is its small composition with these lexical-scope implementations. It is not claimed as a new proof-theoretic idea.

**Theorem 2.1** (Equality of accepted certificates)**.** *For each of the two matched reference/candidate pairs, the same finite JSON certificates reach the accepting verdict in idealized Python execution, ignoring physical time, memory, and stack limits.*

*Proof.* All parsed data are ordinary JSON values and built-in containers; there are no user-supplied operator overloads, callbacks, or threads. The moved precheck reads the instruction, current goal, and local opening function. It neither consumes the certificate stream nor mutates the data, retained context, or scope floor. Its only caller-local change is replacement of the current goal.

The major check receives exactly the same arguments $m, g, k$ in either program. It has its own local floor and local context binding, and it does not refer to the caller's goal. The byte version extends contexts by list concatenation; the AST version uses tuple concatenation. Neither changes its caller's context. The relevant shared effect is consumption of the certificate stream, which the pure precheck never accesses.

After a successful major check, its return value is `None`. For the recursive version this follows from its bare or implicit return; for the loop version it follows from falling out of the loop. Thus the leading operand in the existing `or` expression is false and evaluation continues with exactly the old expression. The following state, goal, scope floor, and remaining input stream are the same.

Induct on a finite successful checking tree. Reorder the precheck and major call at its root, apply the induction hypothesis to recursive major calls and continuations, and obtain the same accepting execution. The argument works in both directions. If either local computation raises an error, it is not swallowed by this transformation and the final verdict cannot be reached. The first exception or rejection latency may change, which is immaterial to the stated acceptance contract. □

The theorem transfers the existing mathematical soundness and completeness claim to the candidates; it does not independently reprove every invariant of the inherited permissive certificate decoder. In particular, the AST profile retains its existing empty-string and empty-object truth aliases. These are recorded in the tests, rather than silently changing the input language during comparison.

## 2.3 Executable evidence and provenance

The delivered `verify.py` checks 49 inherited certificates and 14 boundary/alias cases against all four complete programs, under both ordinary and optimized Python: 504 command-line runs. The outcome was zero mismatches. Six additional mutation-control runs deliberately introduced three known unsound changes: a shared mutable context, an incorrect induction selector, and allocating a fresh parameter before opening its witness. Each corresponding false-proof control was accepted

by the deliberately broken program, while the actual programs rejected it. The complete per-run ledger is `tests/results.json`.

The inherited certificates exercise arithmetic computation, equality transport, literal-equation induction, arbitrary-term $Q$ specialization, compound cases, and nonvacuous existential witnesses. Negative cases cover failed majors, false equations, forbidden compound induction, escaping parameters, context leaks, and numeric aliases. Finite tests are evidence for implementation fidelity; the universal argument is the source theorem above and the inherited calculus proof.

An exact normalized-AST screen covered 4,532 Python file occurrences, representing 549 distinct raw sources, in the pinned Research tree. Neither candidate had an exact match, and no file failed to parse. This establishes only absence of an exact normalized program in that screened corpus. It does not establish conceptual novelty; the prior prevalidation work has already been acknowledged.

### 2.4 Complete byte-profile source

The following listing is the complete 528-byte normalized module. Line wrapping is typographical and is not part of the measured file.

```
import json

def c(a, g=[[]], k=0):
    i = lambda a, t=0j, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a
     < d else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m in ([s, u], not s and [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a
    := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

## 3 How to use the package and what remains open

```
python3 measure.py
python3 verify.py
python3 code/checker_bytes.py < tests/example_accept.json
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The first command reproduces the complete-module metrics. The second recreates the executable validation ledger. The third runs an actual certificate through the new byte profile. The report's TeX source is self-contained: its mathematical sections and displayed checker source are embedded, so it does not require fetching a bibliography, repository, or other document. TeX packages and a Python installation are the ordinary build dependencies. The ZIP includes the separate section sources for convenient review as well.

**Further research questions.**

1. Can the matrix-sensitive closure in the next section be implemented with less total checking work than the existing mode test? Its source and target instantiations, iterator construction, root test, and public driver must all be included in the comparison.
2. Can a graph interpretation recover arbitrary initialization from a smaller numerical syntax while internally proving totality, functionality, induction transfer, and both interpretation composites? The zero-set obstruction below excludes a direct term compiler, but does not settle this more general question permitted by the request.
3. Can the mixed-closure translation and the same-category congruence lemma be formalized in the repository's existing Lean framework? A formalization should expose the syntax-sensitive meaning of closure and forbid cross-category congruence explicitly.
4. Is there a mathematical lower bound for a carefully fixed class of standalone proof presentations? A census of existing source files is not such a lower bound, and unrestricted "simplest" remains dependent on the chosen presentation language and size convention.

The next two sections contain the new mathematical alternatives and obstruction. The appendices reproduce the established native calculus and arithmetic bridge with attribution. The new arguments are written proofs reviewed for circularity and scope conditions; no new Lean or Rocq verification is claimed.

# 4 An alternative with deterministic, matrix-sensitive closure

This section gives a different logical presentation of the same arithmetic. It removes the independent case/progress choice from introduction by changing the meaning and elimination of an atomic closure. It does *not* claim a smaller complete checker: the additional arithmetic in elimination must be counted. In particular, the existing witness-selected introduction already gives a deterministic introduction by another method. The result here is an alternative bi-interpretation theorem, not an optimality theorem.

The comparison is with the native scalar core in the pinned `ConciseIPRA` snapshot. Its existing progressive-binder proposal uses progressive semantics at every matrix and a separate Boolean-cases principle. The construction below uses progressive semantics only at literal equations, retains anchored semantics at compound matrices, and derives numerical cases through the embedding proved below. No literature-wide priority claim is made.

## 4.1 Complete native presentation

Terms and formulas are finite syntax:

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h), \qquad B ::= (s = t) \mid Q_y B \mid D_{x,y} B.$$

Only $u$ is bound in $h$; $n$ and $a$ are outside that binding. The displayed binders avoid capture, and contexts are finite reusable lists of formulas. Put $1 = S0$ and introduce two metalinguistic abbreviations, neither of which adds a term constructor:

$$A(n, t) := I(n, t; u.Su), \qquad d_B(n) := \begin{cases} n, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases} \tag{MC1}$$

"Literal equation" means that the outermost constructor of the fixed matrix is equality. An equation beneath $Q$ or $D$ is compound at that outer level. Numerical substitution preserves this classification. This is a syntax-sensitive presentation, equivalently a grammar with distinct atomic and compound closure forms. It is not a claim that one ordinary logical connective changes meaning under replacement by an equivalent formula. The rules below permit numerical substitution, not unrestricted substitution of formulas for their matrices.

The two computation axioms, available in every context, are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{MC2}$$

The remaining primitive rules are exactly

$$\frac{}{\Gamma \vdash B}\ (B \in \Gamma), \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}, \tag{MC3}$$

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(d_B(n))}{\Gamma \vdash B(A(d_B(n), t))}, \tag{MC4}$$

$$\frac{\Gamma \vdash D_{x,y} B(x, y) \qquad \Gamma, Q_y B(b, y) \vdash F}{\Gamma \vdash F}, \tag{MC5}$$

$$\frac{\Gamma, B(t, d_B(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}. \tag{MC6}$$

In transport, $F$ is any native formula template. In (MC5), $b$ is fresh for $\Gamma$, $F$, and the passive parameters of $B$. In (MC6), $z$ is fresh for $\Gamma$, the witness $t$, and the passive parameters of $B$. The displayed local assumptions are discharged. Call this calculus $\mathcal{C}_{\mathrm{m}}$. There are no additional equality, successor, logical, or induction axioms. As in the inherited core, the designated contradiction is $0 = 1$.

Rule (MC4) has two numerical certificate parameters, $n, t$. For compound $B$, its source is $B(0)$ and its target reduces by a computation equation to $B(t)$. For an equation $E$, it transports progress from $E(n)$ to $E(A(n, t))$. The order of arguments matters: $A(0, t) = t$ is a direct computation, whereas $A(t, 0) = t$ is a derived induction theorem. Reversing those arguments would invalidate the bootstrap proof given next.

## 4.2 Induction and the logical padding operation

Weakening, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible by induction on proofs. For substitution, the test defining $d_B$ is unchanged, and the computation equations are schemas over fixed finite terms. At eigenvariable rules, rename the eigenvariable away from the substituted terms or inserted derivations.

The usual elementary equality rules are derived from computation and transport. For example, if $r = I(0, t; u.0)$, computation proves $r = t$; transporting that equation along itself gives $t = t$. The standard equality motives then give symmetry, transitivity, and numerical congruence.

**Lemma 4.1** (Closure introduction and equation induction)**.** *The following are admissible:*

$$\frac{\Gamma, B(d_B(z)) \vdash B(Sz)}{\Gamma \vdash Q_y B(y)}, \qquad \frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)},$$

*where the second rule requires a literal equation $E$, and the usual eigenvariable conditions apply. Also, for every native matrix $B$,*

$$Q_y B(y), B(0) \vdash B(t). \tag{MC7}$$

*Proof.* For closure introduction, apply (MC6) with an unused witness coordinate and witness 0, then eliminate the resulting $D$ formula. Its opened $Q$ assumption is independent of the fresh witness, so it is also the desired conclusion.

For (MC7), take $n = 0$ in (MC4). Both possible matrices have $d_B(0) = 0$. Computation gives $A(0, t) = t$, and transport gives the stated target. For equation induction, closure introduction applied to the supplied step gives $Q_y E(y)$; use the supplied base and (MC7). No ordinary logical connective or numerical-cases principle is used here. □

**Lemma 4.2** (A raw progress step)**.** *For a literal equation $E$,*

$$Q_y E(y), E(n) \vdash E(Sn).$$

*Moreover, $Q_y B(t, y) \vdash D_{x,y} B(x, y)$ for every matrix $B$.*

*Proof.* Equation induction proves $A(n, 1) = Sn$: the base is a computation, and the successor computation reduces the induction step to congruence of successor. Use (MC4) with the specified source $n$ and with second parameter 1, then transport along this equation.

For the second assertion use (MC6) with witness $t$. For an equation matrix its premise follows from the raw progress step; for a compound matrix it follows from (MC7) at $Sz$. □

We need a carefully restricted congruence fact. If two matrices have the *same root category* (both literal equations or both compound) and their instances are uniformly interderivable, then their $Q$ closures and their $D$ closures are interderivable. For $Q$, use closure introduction: convert its local antecedent to the old one, use Lemma 4.2 in the atomic case or (MC7) in the compound case, and convert the successor back. For $D$, first open its witness and then repeat that construction with the same witness. The reverse proof is identical. This assertion does *not* permit replacing an atomic matrix by an equivalent compound matrix beneath an already present closure. Their closure meanings need not agree.

Define the selector term $C(v, a, b) = I(v, a; u.b)$, with unused $u$, and the gate on formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \quad G_v(Q_y B) := Q_y G_v B, \quad G_v(D_{x,y} B) := D_{x,y} G_v B.$$

Bound variables are first renamed away from $v$. Computation, closure introduction, and the same-category congruence just proved give, by external induction on finite formula syntax,

$$\vdash G_0 B, \qquad G_{Sv} B \dashv\vdash B. \tag{MC8}$$

The first induction uses the theorem for the smaller matrix to supply the successor premise, ignoring its local assumption. The second uses the two selector computation equations at the atomic leaf and same-category congruence at binders. A gate preserves every root category.

**Lemma 4.3** (Compound padding)**.** *Let $r, v$ be fresh for $B$ and put*

$$P(B) := D_{r,v} G_v B.$$

*Then $P(B)$ is compound and $P(B) \dashv\vdash B$, uniformly in every retained context and under numerical substitution.*

*Proof.* Under the retained assumption $B$, introduce $P(B)$ with witness 0. Whether (MC6) supplies $G_z B$ or $G_0 B$ as its local assumption, its successor target $G_{Sz} B$ follows from the retained $B$ by (MC8). Conversely, open $P(B)$, use the theorem $G_0 B$ as the base of its $Q$ closure, and specialize at 1 by (MC7). The result $G_1 B$ gives $B$. Fresh bound coordinates make these constructions commute with capture-free numerical substitution, up to harmless renaming. □

### 4.3 An embedding of the existing native scalar core

Let $\mathcal{C}$ be the existing Eq/$Q$/$D$ core: its $Q$ elimination is (MC7), and it has separate case and literal-equation progress introductions for $D$. Define a translation that leaves all terms unchanged and pads every translated matrix:

$$(s = t)^\dagger = (s = t), \qquad (Q_y B)^\dagger = Q_y P(B^\dagger), \qquad (D_{x,y} B)^\dagger = D_{x,y} P(B^\dagger). \tag{MC9}$$

All padding coordinates are fresh. Contexts are translated pointwise. Every translated binder has a compound matrix, even when its original matrix was an equation.

**Theorem 4.4** (Proof translation)**.** *If $\Gamma \vdash_{\mathcal{C}} B$, then $\Gamma^\dagger \vdash_{\mathcal{C}_{\mathrm{m}}} B^\dagger$. The transformation is effective on finite derivations.*

*Proof.* Induct on the derivation. Hypotheses, computation, and transport are preserved; the last assertion uses commutation of the translation with numerical substitution. Old $Q$ elimination first pads its base proof, specializes the new compound closure, and unpads its result. Old $D$ elimination opens exactly the translated old $Q$ assumption.

For old case introduction, the translated old premise starts from $B^\dagger(t, 0)$ and gives $B^\dagger(t, Sz)$. The new introduction has compound matrix $P(B^\dagger)$: unpad its local base assumption, insert the translated premise by cut, and pad the conclusion.

For old progress introduction with equation $E$, the translated premise is $\Gamma^\dagger, E(t, z) \vdash E(t, Sz)$. Introduce the translated conclusion $D_{x,y} P(E(x, y))$ by the new compound case rule. Under its base $P(E(t, 0))$, unpad to $E(t, 0)$. Lemma 4.1, with a separately fresh induction parameter, gives $E(t, Sz)$; pad this result to finish the premise. This is the only old rule needing induction in the translation. □

The inherited scalar core reconstructs ordinary intuitionistic first-order logic, numerical coverage, separation, and contextual equation induction. Theorem 4.4 therefore imports their proofs and their logical introduction/elimination rules for translated formulas. Those rules remain usable with arbitrary additional retained new formulas: their proof transformations only inspect the active assumptions, and weakening and cut were proved above. This observation suffices for the composite proof below; it does not assume unrestricted cross-category binder congruence.

### 4.4 Ordinary interpretation and both composites

Let $T_{\mathrm{eq}}$ be the ordinary scalar realization of the existing proposal: intuitionistic first-order equality logic over the iterator terms, with (MC2), numerical coverage, separation $(0 = 1) \to \bot$, and induction for literal equations with side assumptions. The inherited arithmetic bridge identifies $T_{\mathrm{eq}}$ with full first-order iPRA. Our interpretation $\rho$ of the new calculus in $T_{\mathrm{eq}}$ leaves terms and equality unchanged and sets

$$(Q_y B)^\rho = \begin{cases} \forall y \big(B^\rho(y) \to B^\rho(Sy)\big), & B \text{ is a literal equation}, \\ B^\rho(0) \to \forall y\, B^\rho(y), & B \text{ is compound}, \end{cases} \qquad (D_{x,y} B)^\rho = \exists x\, (Q_y B)^\rho. \tag{MC10}$$

Thus the atomic reading is genuinely changed; it is not the old anchored reading with a proof choice silently removed.

**Lemma 4.5** (Soundness of the new rules)**.** *The interpretation $\rho$ sends new derivations to $T_{\mathrm{eq}}$ derivations.*

*Proof.* Only introduction and elimination merit comment. Atomic introduction is directly a proof of the displayed progressive statement with a specified existential witness. Compound introduction proves its anchored statement by numerical coverage: zero is its retained base, and successors are its supplied premise.

For atomic elimination, equation induction first proves in $T_{\mathrm{eq}}$

$$A(n,0) = n, \qquad A(n,Sr) = SA(n,r).$$

The first is induction on $n$. The second is induction on $n$ with $r$ as a passive parameter; both steps reduce by computation and successor congruence. Under the progressive premise and $E(n)$, equation induction on $r$ now proves $E(A(n,r))$. Taking $r = t$ gives (MC4). The inducted formula is still one literal equation, not an arbitrary compound formula. Compound elimination is ordinary anchored specialization, since $A(0,t) = t$. The other rules have their immediate ordinary meanings. □

Let $J$ be the existing native interpretation of $T_{\mathrm{eq}}$ in $\mathcal{C}$, including its ordinary logical macros. Define the reverse interpretation $K$ by $K(\varphi) = J(\varphi)^{\dagger}$. Its proof translation follows from Theorem 4.4.

**Theorem 4.6** (Whole-domain bi-interpretation)**.** *The interpretations $\rho, K$ are a bi-interpretation between $\mathcal{C}_{\mathrm{m}}$ and $T_{\mathrm{eq}}$. Both numerical domains are whole, equality is actual equality, and both numerical composites have the identity graph $x = y$. Consequently the new calculus is bi-interpretable and equiconsistent with iPRA.*

*Proof.* First, for every old native formula $F$, $\rho(F^{\dagger})$ is provably equivalent to the old ordinary reading of $F$. Prove this externally by induction on $F$. Every translated matrix is compound; hence its new closure has the old anchored reading. Lemma 4.3, transported by Lemma 4.5, removes the inserted padding. The old ordinary composite theorem therefore gives $\rho(K\varphi) \leftrightarrow \varphi$ for every ordinary formula $\varphi$.

For the new composite, prove externally on native formulas that $B \dashv\vdash K(B^{\rho})$. Equality is unchanged. Suppose the assertion is established for a matrix $B$ and write $\widehat{B} = K(B^{\rho})$; this is a formula in the image of $K$.

If $B = E$ is atomic, a new $Q_y E$ supplies the raw step $E(z) \to E(Sz)$ by Lemma 4.2. Convert its two instances to their hats and use the imported ordinary implication and universal introductions. This gives $K(\forall y(E^{\rho}(y) \to E^{\rho}(Sy)))$. Conversely, assume that imported progressive formula. Under $E(z)$, convert to $\widehat{E}(z)$, eliminate the imported universal and implication, convert the result back to $E(Sz)$, and use derived closure introduction. These are exactly the two directions of the atomic $Q$ composite.

For compound $B$, start instead with an assumption $\widehat{B}(0)$, convert to $B(0)$, specialize the given new $Q_y B$ at an arbitrary fresh parameter, and convert back to its hat. Imported implication and universal introduction yield the anchored ordinary formula. Conversely, under the base $B(0)$, that imported formula supplies $B(Sz)$ after the same conversions; compound closure introduction finishes. No congruence is applied across the potentially different roots of $B$ and $\widehat{B}$.

Finally, for $D_{x,y}B$, open its fresh witness, apply the just-proved $Q$ equivalence for the matrix $B$, and introduce the imported existential. Conversely, eliminate that existential at a fresh witness, convert its matrix back to $Q_y B$, and use the existential injection derived in Lemma 4.2. All ordinary logical

rules here act on image formulas; arbitrary additional new assumptions are merely retained. This completes the formula composite proof.

Both interpretations leave every scalar term and numerical variable unchanged, so their numerical composites are literally the identity. Composition with the inherited $T_{\mathrm{eq}}$–iPRA bi-interpretation yields the assertion. The contradiction $0 = 1$ is unchanged, giving equiconsistency as well. □

**Assessment.** The result removes the independent introduction mode and exposes a different distribution of arithmetic and logic: literal-equation closures are progressive, while compound closures are anchored. The padding embedding proves that this does not restrict ordinary intuitionistic arithmetic. However, elimination now builds an iterator term and supplies an additional numerical parameter. Counting only the single introduction schema would conceal that cost. No measured improvement, proof-assistant verification, or minimality claim is made for this alternative.

## 5 Why fixing the initial state to zero is not a free simplification

An initialized iterator carries three pieces of data: its counter, initial state, and update. A tempting deletion is to fix the initial state to zero. The following standard-model obstruction explains why the resulting language cannot simply inherit the current term compiler or selector-based logical bootstrap. It treats a different operation from the positive iterator $h^{n+1}(a)$ studied in the reviewed `checker_and_barriers` and `prevalidated_elimination` packages: here zero iterations are allowed, but the seed is always zero. This observation is new relative to those reviewed analyses; no literature-wide priority or new iPRA presentation is claimed.

**Two precise candidate languages.** In the *scalar* language the numerical terms are

$$t ::= x \mid 0 \mid S(t) \mid R(n; u.h), \qquad R(0; u.h) = 0, \qquad R(Sn; u.h) = h[R(n; u.h)/u]. \tag{ZS}$$

Only the update $h$ binds $u$; the counter $n$ does not. In the *counter-aware* variant the last constructor is $R(n; k, u.h)$ and its equations are

$$R(0; k, u.h) = 0, \qquad R(Sn; k, u.h) = h[n/k, R(n; k, u.h)/u]. \tag{ZC}$$

Both $k$ and $u$ are bound only in $h$. Every term is finite staged syntax; there is no function-variable sort or uniform evaluator. All assertions below concern the indicated standard interpretation in $\mathbb{N}$, with $S(n) = n + 1$. Passive parameters are held fixed during a run. No equality test, conditional operator, additional arithmetic function, or conversion from formulas into numerical values belongs to either grammar.

**Monotonicity theorem.** *Every term of either language denotes a coordinatewise nondecreasing function of all its free numerical variables.*

**Proof.** Use external structural induction on the finite syntax. Variables, zero, and successor satisfy the assertion. For the recursion clause, the induction hypothesis makes the counter and the update nondecreasing in all their arguments.

For a fixed parameter assignment $\rho$, write $a_j^\rho$ for the state after $j$ updates. In the counter-aware case,

$$a_0^\rho = 0, \qquad a_{j+1}^\rho = h(j, a_j^\rho, \rho).$$

The state sequence is nondecreasing: $a_0^\rho \le a_1^\rho$ because zero is least, and from $a_j^\rho \le a_{j+1}^\rho$ monotonicity of $h$ in both its counter and state gives

$$a_{j+1}^\rho = h(j, a_j^\rho, \rho) \le h(j+1, a_{j+1}^\rho, \rho) = a_{j+2}^\rho.$$

For parameter assignments $\rho \le \sigma$, induction over the common update index gives $a_j^\rho \le a_j^\sigma$. The counters satisfy $n(\rho) \le n(\sigma)$, so

$$a_{n(\rho)}^\rho \le a_{n(\rho)}^\sigma \le a_{n(\sigma)}^\sigma.$$

This proves the recursion clause. The scalar case is the same argument with the update's counter argument omitted. It also covers a free variable occurring both in the counter expression and as a passive parameter of the update. □

**Exact zero-profile theorem for the scalar language.** For a scalar term $t(x_1, \dots, x_r)$, put $Z(t) = \{\vec{x} : t(\vec{x}) = 0\}$. *Its zero set is either the whole domain or a union of coordinate zero-hyperplanes:*

$$Z(t) = \mathbb{N}^r \quad \text{or} \quad Z(t) = \bigcup_{j \in J} \{\vec{x} : x_j = 0\} \qquad (J \subseteq \{1, \dots, r\}). \tag{Zero profile}$$

The union may be empty. A profile can be computed by structural recursion without evaluating the represented function on unbounded inputs.

**Proof.** Zero has the whole-domain profile, a variable has its one coordinate hyperplane, and a successor term has the empty profile. For the remaining constructor, fix the passive parameters and let $b = h(0)$. If $b = 0$, every iterate from zero is zero. If $b > 0$, monotonicity of $h$ gives $h(v) \ge h(0) = b > 0$ at every natural state $v$; consequently every positive iterate is nonzero. Thus, pointwise,

$$R(n; u.h) = 0 \quad \iff \quad n = 0 \ \vee \ h[0/u] = 0. \tag{Zero recurrence}$$

Hence $Z(R(n; u.h)) = Z(n) \cup Z(h[0/u])$. The update is a proper syntax subterm. Its previously computed profile, with $u$ set to zero, becomes the whole domain if it includes the $u = 0$ hyperplane, and otherwise keeps the same passive-coordinate hyperplanes. Union with the counter's profile preserves the displayed classification.

Conversely, every displayed kind of zero set is realized by a scalar term. Zero and $S0$ realize the whole and empty sets. The constant-update gate $G(c, a) = R(c; u.a)$ satisfies $Z(G(c, a)) = Z(c) \cup Z(a)$, so nested gates realize any prescribed finite union of coordinate hyperplanes. This converse classifies zero sets, not all numerical functions sharing a zero set. □

**Consequences for the proposed deletion.** In the scalar language no term denotes predecessor: the unary function $\max(n - 1, 0)$ has zero set $\{0, 1\}$, whereas a unary term has zero set only $\varnothing$, $\{0\}$, or $\mathbb{N}$. Nor does a binary term denote ordinary addition, whose zero set is the singleton $(0, 0)$; that set is neither a union of the two coordinate hyperplanes, nor empty, nor the whole domain.

In *both* languages the function equal to one at zero and zero at every positive input is unavailable, because it is decreasing from zero to one. The initialized selector

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b$$

is therefore unavailable too: specializing $a = 1, b = 0$ would give that zero-test. In particular, there is no term compiler that recovers every initialized iterator from either proposed zero-start grammar

on the same numerical domain with the standard readings of zero and successor. This conclusion already follows from a constant-update instance, before questions about pairing or general primitive recursion arise.

The extra old-counter argument makes a real difference. In the counter-aware language

$$P(n) := R(n; k, u.k) \quad \text{satisfies} \quad P(0) = 0, \qquad P(Sn) = n.$$

Thus predecessor is a term there, and the scalar zero-profile theorem does not extend to that language. Its proof fails exactly because the update can change with the counter: the first update can be zero while a later update is positive. The monotonicity theorem still applies, and still rules out the selector and the zero-test. No claim about term-definability of addition in the counter-aware language is needed or established here.

**What this does not prove.** These are standard-model, same-domain *term-definability* obstructions. They do not rule out function graphs using quantified formulas, definable subdomains, tuple domains, quotient equality, or some other bi-interpretation permitted in the question. Such a proposal would need its own proof of totality and functionality of the interpreted operations, transfer of the restricted induction schema, closure under newly defined updates, and definable isomorphisms for both composites. The argument above establishes none of those positive requirements and is not presented as an impossibility theorem for all simpler axiomatizations of iPRA.

# Attribution and dependency boundary for the appendices

The following five sections are reproduced from the pinned repository's `Research/certified_zero/reference/` directory, with its MIT-0 license. They supply the existing native logical basis and scalar arithmetic bridge. They are not new contributions of this note. In particular, the phrase "new" in an inherited section refers to its own original comparison, not to the present work. The first appendix uses conditional context clauses; the main text uses the equivalent explicit $Q$ notation. Their original labels and mathematical content have been retained, while citations are resolved to the specific pinned source files below.

# A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

## A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only $h$ binds the state variable $u$; $n$ and $a$ do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables $x, y$ in a paired formula are distinct. An ordinary assumption $F$ is the constant clause $\langle H; v.F\rangle$, where $v$ is absent from $F$ and $H$ is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here $\varepsilon$ is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when $B$ has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter $z$ is fresh for $\Gamma$, the witness $t$, and every fixed parameter of $B$. In DE the eigenparameter $a$ is fresh for $\Gamma$, the conclusion $F$, and every fixed parameter of $B$. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t, 0), [B(t, z)]_{\mathrm{at}} \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}. \tag{Old DI}$$

The bracket supplies $B(t, z)$ exactly at a literal equation.

## A.2 Exact equivalence and the unused-witness construction

**Lemma A.1** (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

*Proof.* Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

**Theorem A.2** (Identity-language equivalence)**.** *For every retained clause context $\Gamma$ and native formula $F$,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

*The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.*

*Proof.* Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x, y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t, 0), E(t, z) \vdash E(t, Sz).$$

The outer new inference will be case DI with witness $t$. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in $\pi$ to a fresh $v$. Choose a new variable $u$ absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate $u$ is deliberately unused.

Apply DE to this paired formula. Because $u$ is unused, the opened clause is exactly

$$\langle E(t, 0); y.E(t, y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t, 0)$ supplies its antecedent. Use at $Sz$ gives $E(t, Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x, y)$ with witness $t$. The original premise proof $\pi$ has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x, y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original $t$. The construction instead keeps $t$ as a fixed parameter of $E(t, y)$ and binds only the unused coordinate $u$.

**Proposition A.3** (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of $H$ followed by Use. If an old derivation has $N$ nodes and $r$ atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

*nodes.*

*Proof.* Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of $H$ needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

### A.3 Induction and numerical cases separate cleanly

**Lemma A.4** (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

*for every equation $E$, with the usual freshness condition.*

*Proof.* Choose an unused witness coordinate $u$. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at $t$. Close DE. □

**Lemma A.5** (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

*with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under* $\mathsf{D}$ *are derivable without using the progress choice.*

*Proof.* For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at $t$ finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness $a$ and introduce the new one by case DI at $a$. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

### A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with $u$ unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where $E$ is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A,B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A,B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A,B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; $u$ is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A,B)$ and the implication gives $F$.

Given $A$ and $B$, selector endpoints and NC give $M_v(A,B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give $B$ and $A$, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains $x$; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0,s,t) = C(1,s,t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

### A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let $T_{\mathrm{eq}}$ have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh $a$; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary $y$. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh $a$. Introduce the paired formula with witness $a$ by case DI: the retained base, supplied implication, and universal elimination at $Sz$ give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies $T_{\mathrm{eq}}$ with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

### A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

# B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let $T_I$ be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from $T_I$.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of $T_I$. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused $v$: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

## B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply $P$ to obtain $Px = y$ and substitute.

2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.

3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let $\pi, L, H$ be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

**Theorem B.1** (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization $T_I$ and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

*Proof.* Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

# C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [5] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, $S$, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only $u$. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

### C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of $C$ is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on $b$; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of $N$ is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of $Z$. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of $N$; the second follows by one computation. In the third, increasing $n$ adds three to the left argument and therefore applies $Z$ once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing $M$ as remainder modulo three.

### C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next $G$ state is $G(n)+1$; its $M$ value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next $G$ state is $G(n) + 2$; two computations of $M$ change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of $G$.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain $u$ when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

### C.3 A comparator from scalar iteration

For every fixed update $h$, equation induction on $n$, keeping $m, a$ and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies $h$ to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update $h$, equation induction on $y$, with its initializer $a$ and all parameters of $h$ fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same $h$ to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays $a$ throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for $H$. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus $L$ has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

### C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in $B_m$ records a fixed parameter; $B_m$ is an open term abbreviation, not a function variable.

Numerical cases on $e$ prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on $e$, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on $t$, with $m$ fixed, proves

$$W(St, Sm) = SW(t, m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed $m$.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on $t$, with $m$ fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

## C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying $P$ to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

# D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [6]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n,a,b) = I(n,a;u.b)$, addition is $x+y = I(y,x;u.Su)$, and truncated subtraction is $d(x,y) = I(y,x;u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

## D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let $\mathsf{PRE}$ contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions $P$ and $C$ with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0,a,b) = a, \qquad C(Sx,a,b) = b. \tag{17}$$

Write $1 = S0$. In addition to $\mathsf{PRE}$, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n,\vec{p})$:

$$\forall \vec{p}\,\left[(E(0,\vec{p}) \wedge \forall n(E(n,\vec{p}) \longrightarrow E(Sn,\vec{p}))) \longrightarrow \forall n\, E(n,\vec{p})\right]. \tag{20}$$

Here the schematic expression $E$ is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where $z$ is absent from $\Gamma$ and the passive parameters, discharge the second premise's equation and universally introduce $z$. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in $\Gamma$.

### D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

**Constructor laws.** Successor is injective: applying $P$ to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

**The sign test.** For every $u$,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all $u, v$,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to $v$, so this identity again needs no induction.

**The countdown test and the notation $W$.** For every $u$,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named $W$ with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every $u$ already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are $Pu$. Thus $K$ and $W$ are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

### D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on $y$. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold $x$ fixed and induct on $y$. At zero both sides are $Sx$. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold $x$ fixed and induct on $y$. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying $P$ to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on $a$ for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

### D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all $x, y$:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold $x$ fixed and induct on $y$. At $y = 0$, subtraction gives residual $x$, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at $y$ and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for $r'$.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for $r$. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since $r'$ is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

**Reflection at residual 1.** We have, for arbitrary $a, b$,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying $P$ gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict $a$ or $b$ to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

### D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of $A$ proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if $A$ held, the already established implication $A \to b_A = 0$ would contradict separation.

**Atomic formulas and falsity.** For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

**Conjunction.** Split $b_A = 0 \vee b_A = 1$. In the zero branch, $A$ holds and $b_{A\wedge D} = b_D$. Its range follows from the range of $b_D$, and its being zero is equivalent to $D$, hence to $A \wedge D$ under the known $A$. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

**Disjunction.** In the branch $b_A = 0$, the formula $A$ holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of $b_D$. If its value is zero, then $D$ and hence $A \vee D$. Conversely, from $A \vee D$, the $A$ alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the $D$ alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

**Implication.** In the branch $b_A = 0$, the formula $A$ holds and $b_{A\to D} = b_D$. Its range follows from the range of $b_D$. If its value is zero, then $D$, which gives $A \to D$. Conversely, $A \to D$ and the known $A$ give $D$, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume $A$; this contradicts $\neg A$, and explosion gives $D$. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

### D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context $\Gamma$ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on $z$. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term $t$. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

**Completion theorem.** In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives $\mathsf{Sep}$. Quantifier-free induction proves $\mathsf{Cov}$: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

## E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [7]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

### E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \leq b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter $n$, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments $v$ exactly when $(v+1)^2 \leq n$, and otherwise retains it. QFI in $i$, with $n$ fixed, proves

$$R_n(i) \leq i, \qquad R_n(i)^2 \leq n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals $i$, and the new state equals $Si$; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \leq n < (q(n)+1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed $n$ during the induction.

Now define

$$\pi(a, b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of $\pi$ is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

## E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier $g, h$ already have scalar translations $g^*, h^*$. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every $p_i$ fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for $f^*$ internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

### E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

## References

[1] Vladimir Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt repository, pinned commit `8662bc0a00f4e77e482f7d141d7a4f919f70f9aa`. Project README and proposal inventory. Accessed for this note on 10 November 2026 (America/Los_Angeles).

[2] ProveIt, *Neutral scope*, same snapshot. Research package; in particular `code/checker.py`, `report.tex`, and the public certificate protocol and scope-invariant analysis.

[3] ProveIt, *Coverage and kernel*, same snapshot. Research package; in particular `code/checker_ast.py`, `theory/kernel.tex`, and the inherited certificate/control collection.

[4] ProveIt, *Prevalidated elimination*, same snapshot. Research package; the antecedent idea of pure template validation before checking the major premise is credited to this package.

[5] ProveIt, *Internal scalar predecessor bootstrap*, frozen reference `certified_zero/reference/scalar_predecessor.tex`, same snapshot. Pinned source; reproduced in this note with its existing attribution.

[6] ProveIt, *Equality reflection*, frozen reference `certified_zero/reference/equality_reflection.tex`, same snapshot. Pinned source; reproduced in this note with its existing attribution.

[7] ProveIt, *Pairing, primitive recursion, and internal numerical composites*, frozen reference `certified_zero/reference/primitive_recursion.tex`, same snapshot. Pinned source; reproduced in this note with its existing attribution.